

LC 975 - Odd Even Jump

© Authored by Arpit Raj, LNMIIT Jaipur | August 14, 2026

Sprinklr, Amazon

Problem Understanding

- Any starting index is called a **good starting index** if we can reach end of array.
- Output:** No. of good starting index.
- In the **oddth jump** one can go from `i` to `j` if `arr[i] <= arr[j]` where `arr[j]` is just greater than `arr[i]`.
- In the **eventh jump** one can go from `i` to `j` if `arr[i] >= arr[j]` where `arr[j]` is just smaller than `arr[i]`.

Core Idea & Logic

IDEA

- `odd[i]` = can reach end if next jump is odd
- `even[i]` = can reach end if next jump is even

Transitions: (transitions of odd and even are alternate)

- `odd[i] = even[oddnnext[i]]`
- `even[i] = odd[evennext[i]]`

For odd jump: `arr[i] <= arr[j]`

Can't use `upper_bound` as it gives strictly >

So we use `lower_bound`.

For even jump: `arr[i] >= arr[j]`

So we use `upper_bound` and `(it--)`.

→ **FOR TIE BREAKERS:** We need to store indexes with value too.

C++ Implementation

```
class Solution {
public:
    int oddEvenJumps(vector<int>& arr) {

        int n = arr.size();
        vector<int> odd(n, 0);
        vector<int> even(n, 0);

        map<int, int> mp; //value to smallest index with that value;
        //so we will process from right to left , because left is
        automatically smaller
        //than any stored index in map , to bada index aise hi replace
        ho jaayega

        odd[n-1] = 1;
        even[n-1] = 1;

        mp[arr[n-1]] = n-1;

        //last index pe already phuch skte

        int ans = 1;

        for(int i = n-2; i >= 0; i--) {

            //for odd jump
            auto it = mp.lower_bound(arr[i]);

            if(it != mp.end()) {
                int next = it->second;
                odd[i] = even[next];
            }

            //for even jump
            it = mp.upper_bound(arr[i]);

            if(it != mp.begin()) {
                it--;
                int next = it->second;
                even[i] = odd[next];
            }
        }
    }
}
```

```
        mp[arr[i]] = i;
        //isliue right to left ja rha tha
        //left wala guarenteed smaller indexx hoga replace kr dega

        if(odd[i]) ans+
+; //pehla jump to odd hoga , isliye sirf odd ke liye
    }
    return ans;
}
};
```